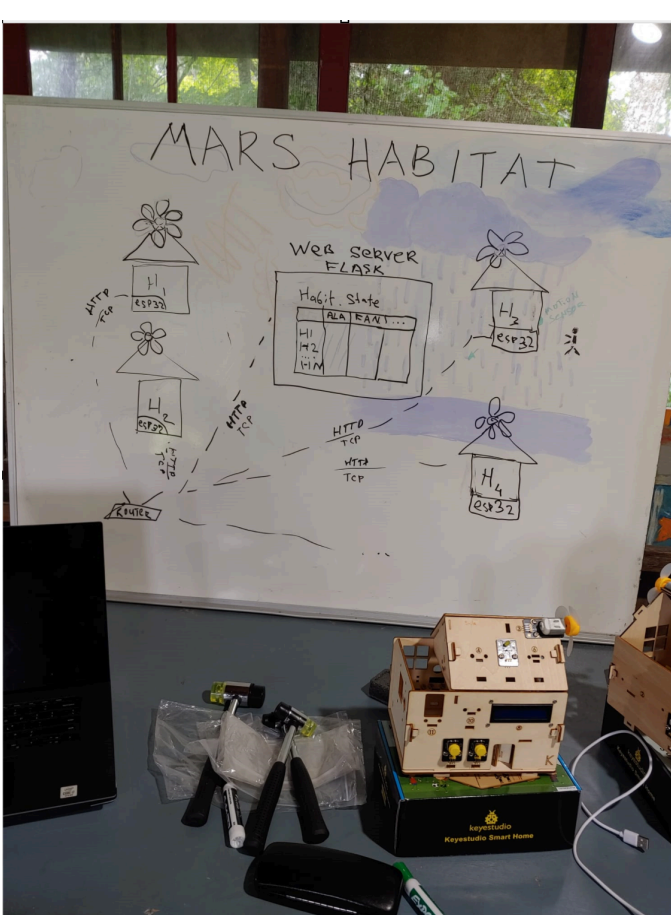


Day 1



What does it take to build a smart house? Smart city? Smart planet?

What are components of the smart house?

What is IoT device? What is a sensor? Actuator?

What is WiFi? What is a server?

Semilab challenge:

- add RFID "module" to your smart house
- Turn the light, the fan using RFID
- Display Different messages on LCD screen based on diff cards/tags

Grand challenge: disconnect all Martians from Habitat Control Center

Day 1:

End-to-End demo, Key concepts - recap from the previous semilab, RFID hello world test

Day 2:

RFID in depth: [RFID theory](#)

Exercise: use different RF tags to control led, fan, buzzer locally

Day 3:

Web interface: add Super User access to existing web app, test locally: assign different RF tags different permissions within the webapp:

- Regular user, User Manager, Admin

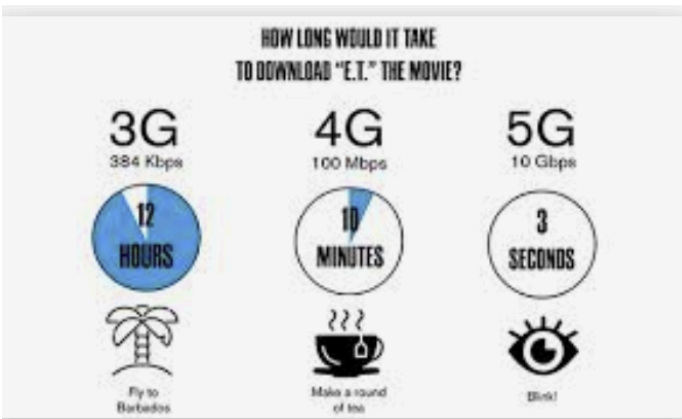
Day 4:

- Add API to existing web framework: based on specific RFID disable or enable remote sensors

Day 5:

- Challenge: TBD

WiFi



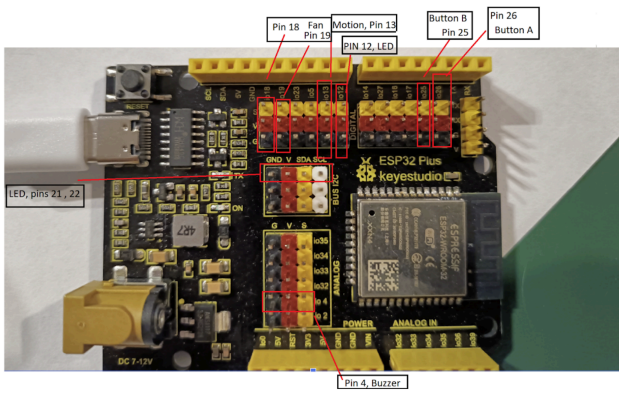
3g:

- Relies on large cell towers spaced far apart.
- Uses frequencies in the 800 MHz to 2.1 GHz range.
- Core network based on circuit-switched technology (for voice) and packet-switched technology (for data).

5g:

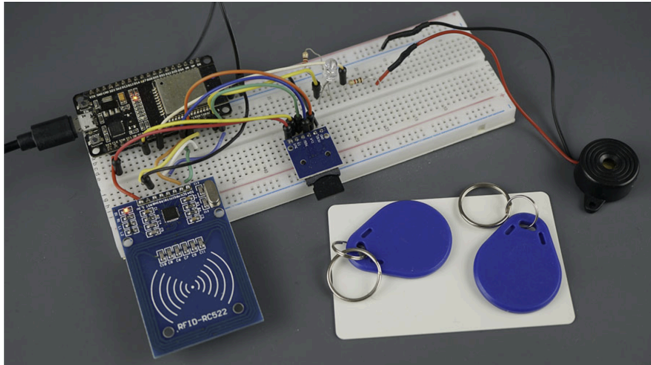
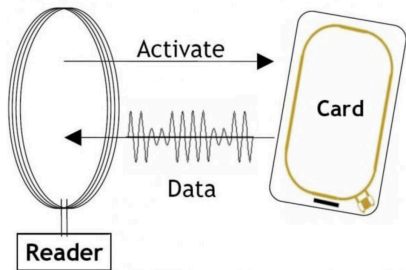
- Requires a dense network of small cells due to higher frequencies (up to 100 GHz), which have shorter ranges.
- Utilizes millimeter waves, which provide higher speeds but are easily obstructed by buildings and trees.
- Core network is entirely packet-switched and IP-based, integrating with cloud technologies and edge computing.

Hello world: server / app.py



ESP32 microcontroller and a set of sensors, actuators and other cool peripherals like buzzers, LCD displays, buttons and LED lights

Day 2

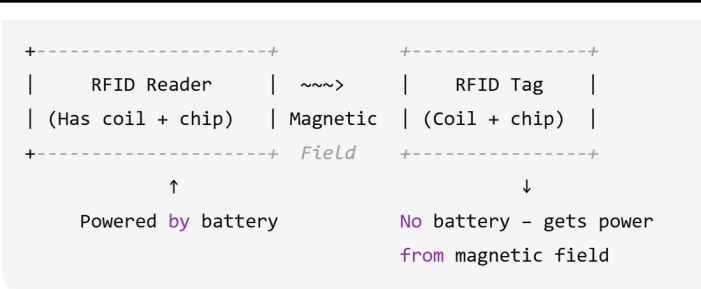


RFID Reader (like the **RC522** module):

- Sends out a **magnetic field**.
- Reads the information from the tag when it comes close.

RFID Tag or Card

- Has **no battery** (it's called a *passive* device).
- Inside it are:
 - A **tiny chip** (stores data UID number) **pre-programmed at the factory**. It's burned into the chip's memory during manufacturing. The UID is usually **read-only**, meaning it **cannot be changed**.
 - An **antenna** (to send and receive signals).



1. The **reader gets the data** and can use it—like identifying who you are or unlocking a door.

Imagine the tag sends:

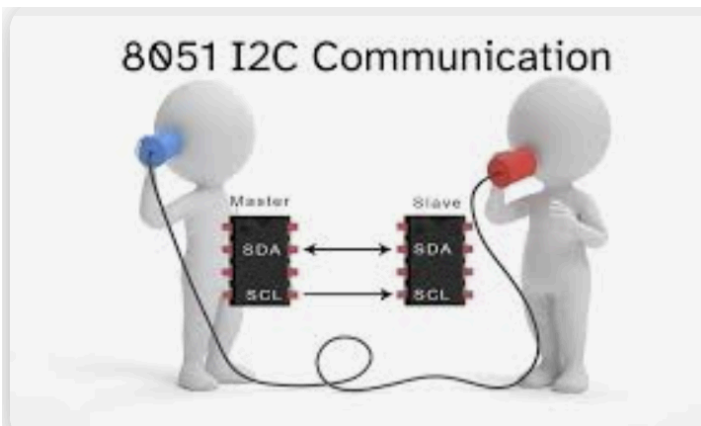
Tag ID = [0xDE, 0xAD, 0xBE, 0xEF]

The reader receives this binary or hexadecimal data and:

1. **Reads the tag ID**
2. **Compares it to a stored list** (e.g., in a database or lookup table)
3. **Identifies the person or object**

0xDE is hexadecimal (base-16) notation.

Day 3.



What Is I²C? (Or how microcontroller communicates with RFID readers)

I²C is a **serial communication protocol** that allows multiple "slave" devices (like sensors, displays, or RFID readers) to communicate with one "master" device (like a microcontroller) over **just two wires**.

Key Concepts of I²C

1. SCL (Serial Clock Line) – Generated by the master

- This line provides the **timing** signal.
- Every bit of data on the SDA line is synchronized with a clock pulse from SCL.
- Without SCL, devices wouldn't know **when** to read or write.

Why SLC is important?

The slave sees a constant or floating data line (SDA) — but has no idea **when** a bit starts or ends.

The slave **can't decode anything**, even if data is technically "being sent."

It's like trying to listen to Morse code without the timing — you'd just hear beeps, but not know when one letter ends and the next starts.

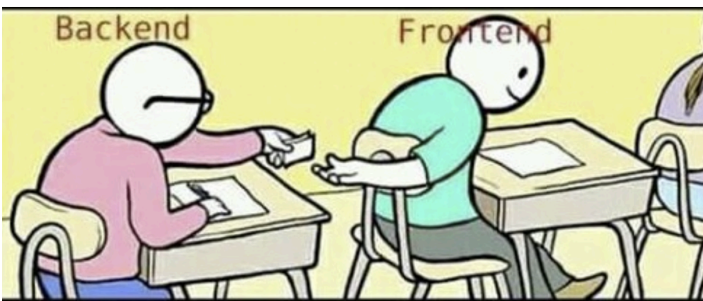
2. **SDA (Serial Data Line)** – Used for sending/receiving bits

- Carries the **actual data** (address, commands, responses, etc.).
- Devices put data on SDA and read from it **in sync with SCL**.

How they work together:

- **SCL** is controlled by the master device and sets the timing for data transfer.
- On each clock pulse on **SCL**, data bits are sent or read over the **SDA** line.
- Both wires are **open-drain/open-collector**, which means devices can only pull the lines low; external **pull-up resistors** keep the lines high when no device is pulling them down.

Rest API



```
User: GET server/api/v1/Joke
Server: Why did the chicken cross the road?
User: GET server/api/v1/Punchline
Server: ...punchline to what?
```

A REST API (Representational State Transfer Application Programming Interface) is a set of rules and conventions for building and interacting with web services. It allows different software applications to communicate with each other over the internet using standard HTTP methods.

RESTful APIs use standard HTTP methods to perform actions on resources. The four most commonly used methods are:

GET: Retrieve a resource or a list of resources.

POST: Create a new resource.

PUT: Update an existing resource or create one if it doesn't exist.

DELETE: Remove a resource

RESTful APIs use standard HTTP status codes to indicate the result of the API request:

200 OK: The request was successful.

201 Created: A new resource was successfully created.

204 No Content: The request was successful, but there is no content to return.

400 Bad Request: The request was malformed or invalid.

401 Unauthorized: Authentication is required and has failed or has not been provided.

404 Not Found: The requested resource does not exist.

500 Internal Server Error: The server encountered an error processing the request.



How we started? (hello world GET method to register)

http://192.168.1.4/smart?serial_number=111111&user_name=Lilia&ip_address=11221212

What are we up to now?

<http://192.168.1.112/smarthouse/v1/houses>

Houses:app.py

Init:

```
self._log("* Event Loop")
self.loop = get_event_loop()

self._log("* IoT Hub Update Timer")
self._iot_hub_update_flag = False
self._iot_hub_timer = Timer(0)
```

```

self._iot_hub_timer.init(
    period=self.config["update_interval_ms"],
    mode=Timer.PERIODIC,
    callback=self._iot_hub_timer_callback,
)
....
if self.exit_code == 0:
    self._iot_hub_register()

    self._lcd_out(self.menu.get_current_content())

    self.loop.create_task(self.event_consumer())

    self._log("Enter event loop")
    try:
        self.loop.run_forever()
    except KeyboardInterrupt:
        self._log("Keyboard interrupt detected. Stopping...")

self._log("Exit event loop")
....

def _iot_hub_timer_callback(self, t):
    """Raise update flag."""
    self._iot_hub_update_flag = True

self._log("Setting up peripheral devices")

```

Device

```

self.button_a = Button(
    name="/in/button_a",
    pin_num=26,
    event_queue=self.event_queue,
    debug=self._DEBUG,
)
....

def _iot_hub_register(self): // step 1
    """Check-in with IoT Hub and provide initial state."""

```

```

self._log("Check-in with IoT Hub")
self._iot_hub_call(
    call_method="POST",
    call_url=f"{self.config.get('api_endpoint')}/houses",
    call_json={
        "unique_id": self.unique_id,
        "ip_address": self.wlan.ip_address,
        "state": self._state,
    },
)

def _iot_hub_keepalive(self):
    """Send keepalive and get latest state from IoT Hub if available."""
    self._log("Send keepalive to IoT Hub")
    response = self._iot_hub_call(
        call_method="PUT",
        call_url=f"{self.config.get('api_endpoint')}/houses/{self.unique_id}/keepalive", # noqa: E501
        call_json={
            "unique_id": self.unique_id,
            "ip_address": self.wlan.ip_address,
        },
    )

    if response.status_code == 202:
        self.alarm.set_trigger(triggered=True, period_ms=4000)

    if response.status_code == 205:
        self._iot_hub_get_state()

def _iot_hub_set_state(self):
    """Send latest state to IoT Hub."""
    self._log("PUSH state to IoT Hub")
    self._iot_hub_call(
        call_method="PUT",
        call_url=f"{self.config.get('api_endpoint')}/houses/{self.unique_id}/state",
        call_json={
            "unique_id": self.unique_id,
            "ip_address": self.wlan.ip_address,
            "state": self._state,
        },
    )

```

```

def _iot_hub_get_state(self):
    """Get latest state from IoT Hub."""
    self._log("PULL state from IoT Hub")
    response = self._iot_hub_call(
        call_method="GET",
        call_url=f"{self.config.get('api_endpoint')}/houses/{self.unique_id}/state",
    )

    try:
        json_response = response.json()
    except ValueError:
        self._log("ERROR: State response not in JSON")
    finally:
        wall_msg_ui = json_response["wall_msg"]
        if self._state["wall_msg"] != wall_msg_ui:
            self._log("* WALL MSG: CHANGED")
            self._state["wall_msg"] = wall_msg_ui
            self._lcd_out(self.menu.get_current_content(), show_wall_msg=True)

        buzzer_active_ui = json_response["buzzer"]["active"]
        if self.buzzer._state["active"] != buzzer_active_ui:
            if buzzer_active_ui:
                self._log("* BUZZER: PLAY")
                self.buzzer.start_melody()
            else:
                self._log("* BUZZER: STOP")
                self.buzzer.stop_melody()
        else:
            self._log("* BUZZER: UNCHANGED")

        fan_active_ui = json_response["fan"]["active"]
        if self.fan._state["active"] != fan_active_ui:
            if fan_active_ui:
                self._log("* FAN: ON")
                self.fan.turn_on(clockwise=True)
            else:
                self._log("* FAN: OFF")
                self.fan.turn_off()
        else:
            self._log("* FAN: UNCHANGED")

        led_active_ui = json_response["led"]["active"]

```



```

if self.led._state["active"] != led_active_ui:
    if led_active_ui:
        self._log("* LED: ON")
        self.led.turn_on()
    else:
        self._log("* LED: OFF")
        self.led.turn_off()
else:
    self._log("* LED: UNCHANGED")

def _iot_hub_report_alarm(self):
    """Send alarm report to trigger other global alarms through IoT Hub."""
    self._log("Send alarm report to IoT Hub")
    self._iot_hub_call(
        call_method="PUT",
        call_url=f"{self.config.get('api_endpoint')}/houses/{self.unique_id}/report_alarm", # noqa: E501
    )

```

Events:

```

async def event_consumer(self):
    """Process events asynchronously."""
    while True:
        if self.event_queue:
            event = self.event_queue.popleft()
            self.event_processor(event)

        if self._iot_hub_update_flag:
            self._iot_hub_update_flag = False
            self._iot_hub_keepalive()

        if self._state_change_remote:
            self._state_change_remote = False
            self._iot_hub_get_state()

        if self._state_change_local:

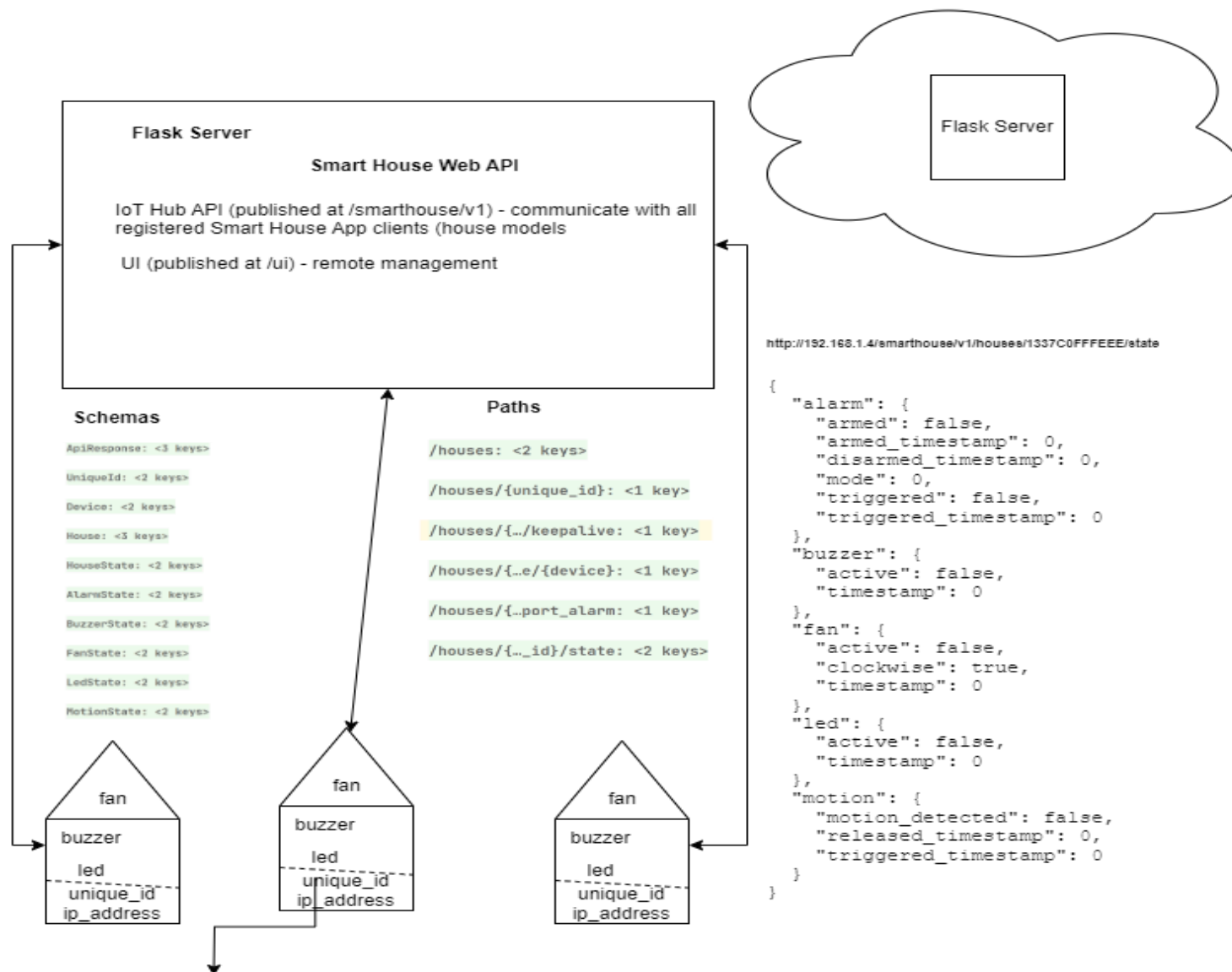
```

```
        self._state_change_local = False
        self._iot_hub_set_state()

    await sleep_ms(100)

def event_processor(self, event):
    """Process event."""
    self._log(f"Got event: {event}")

    if event["source"] == "/in/button_a" and event["state"]["pressed"]:
        self.menu.move_next()
        self._lcd_out(self.menu.get_current_content(), show_wall_msg=True)
```



```
<!DOCTYPE html>
<html>
  <head>...</head>
  <!-- <body onload="fetchHouses();" -->
  <body onload="fetchHouses();setInterval(fetchHouses, 1000);">
    <h2>Smart Houses</h2>
    <table>
      <thead>...</thead>
      <tbody id="house_container"> == $0
        <tr id="1337CAFEC0DE">...</tr>
        <tr id="1337C0FFEEEE">...</tr>
      </tbody>
    </table>
  </body>
</html>
```

Paused in debugger

Unique ID	IP Address	Status	Alarm	Buzzer	Fan	LED	Motion	Last Seen	Time Created	Time Modified	Time Deleted	Wall Message
1337CAFEC0DE	192.168.1.42	Lost	Disarmed	Off	Off	Off	No	2023-07-13 00:01:02	2023-07-13 00:01:02			
1337C0FFEEEE	192.168.1.24	Lost	Disarmed	Off	Off	Off	No	2023-07-13 00:01:02	2023-07-13 00:01:02			

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse | Recorder | Performance insights

Page | Workspace | houses_dashboard.js

```
1 function fetchHouses() {
2   const api_url = "/smart-house/v1/houses"
3
4   fetch(api_url)
5     .then(response => response.json())
6     .then(data => { data = (2) [{...}, {...}]
7       const tableBody = document.getElementById("house_container");
8
9       tableBody.innerHTML = "";
10
11       data.forEach(house => {
12         const row = document.createElement("tr");
13         row.setAttribute("id", house.unique_id)
14
15         const unique_id = document.createElement("td");
16         unique_id.textContent = house.unique_id;
17         row.appendChild(unique_id);
18
19         const ip_address = document.createElement("td");
20         ip_address.textContent = house.ip_address;
21         row.appendChild(ip_address);
```

Paused on breakpoint

▼ Watch

data: Array(2)

0: {global_alarm: false, ip_address: '192.168.1.42', state: {...}, status: 'Lost', timestamp_created: '2023-07-13 00:01:02', timestamp_modified: '2023-07-13 00:01:02', timestamp_deleted: '2023-07-13 00:01:02', wall_message: ''}

1: {global_alarm: false, ip_address: '192.168.1.24', state: {...}, status: 'Lost', timestamp_created: '2023-07-13 00:01:02', timestamp_modified: '2023-07-13 00:01:02', timestamp_deleted: '2023-07-13 00:01:02', wall_message: ''}

length: 2

[[Prototype]]: Array(0)

▼ Breakpoints

Pause on uncaught exceptions

Pause on caught exceptions

▼ houses_dashboard.js

const tableBody = document.getElementById("house_container");

▼ Scope

Local

this: undefined

data: (2) [{...}, {...}]

tableBody: <value unavailable>

Global

```
[
{
  "global_alarm": false,
  "ip_address": "192.168.1.42",
  "state": {
    "alarm": {
      "armed": false,
      "armed_timestamp": 0,
      "disarmed_timestamp": 0,
      "mode": 0,
      "triggered": false,
      "triggered_timestamp": 0
    },
    "buzzer": {
      "active": false,
      "timestamp": 0
    },
    "fan": {
      "active": false,
      "clockwise": true,
      "timestamp": 0
    },
    "led": {
      "active": false,
      "timestamp": 0
    },
    "motion": {
      "motion_detected": false,
      "released_timestamp": 0,
      "triggered_timestamp": 0
    }
  },
  "status": "Lost",
  "timestamp_created": "2023-07-13 00:01:02",
  "timestamp_deleted": "",
  "timestamp_heartbeat": "2023-07-13 00:01:02",
  "timestamp_modified": "",
  "unique_id": "1337CAFEC0DE",
  "update_from_ui": false
},
{
  "global_alarm": false,
  "ip_address": "192.168.1.24",
```

```
{
  "state": {
    "alarm": {
      "armed": false,
      "armed_timestamp": 0,
      "disarmed_timestamp": 0,
      "mode": 0,
      "triggered": false,
      "triggered_timestamp": 0
    },
    "buzzer": {
      "active": false,
      "timestamp": 0
    },
    "fan": {
      "active": false,
      "clockwise": true,
      "timestamp": 0
    },
    "led": {
      "active": false,
      "timestamp": 0
    },
    "motion": {
      "motion_detected": false,
      "released_timestamp": 0,
      "triggered_timestamp": 0
    }
  },
  "status": "Lost",
  "timestamp_created": "2023-07-13 00:01:02",
  "timestamp_deleted": "",
  "timestamp_keepalive": "2023-07-13 00:01:02",
  "timestamp_modified": "",
  "unique_id": "1337C0FFEEEE",
  "update_from_ui": false
}
```

Data Flow:

<http://192.168.1.112/>

Request routed to flask server that has defined path for “/”:

```
@app.route("/")
def hello_world():
    """Serve test page."""
    return render_template("hello_world.html")
```

hello_world.html

```
<body>
<h1>Hello, World!</h1>

<a href="/ui">Click here for the IoT Hub Interface</a>
<br>
<a href="/static/list_houses">Or, go here to see static list of houses for testing</a>
</body>
```

```
@app.route("/ui")
def ui_index():
    """Serve dashboard page."""
    return render_template("ui_index.html", houses=HOUSES)
```

ui_index.html

```
<body onload="fetchHouses();setInterval(fetchHouses, 1000);">
```

https://www.dropbox.com/scl/fo/ou9zo3sb83jilhnnpnqoiu/AGMbEyImgmlyl41jTC4fK2M/4.%20Python%20Tutorials?dl=0&rlkey=s5z0rkeov3p0ezmu4pimson48&subfolder_nav_tracking=1